# 2 - Reading and Writing Data with DataFrames

This demonstration will introduce you to Spark DataFrames by reading data from an external source into a DataFrame and then writing the DataFrame out to another location.

## Objectives

- Read external data into a DataFrame
- Create and inspect DataFrame schemas
- Write the contents of a DataFrame to a specified location

## REQUIRED - SELECT CLASSIC COMPUTE

Before executing cells in this notebook, please select your classic compute cluster in the lab. Be aware that **Serverless** is enabled by default.

Follow these steps to select the classic compute cluster:

1. Navigate to the top-right of this notebook and click the drop-down menu to select your cluster. By default, the notebook will use **Serverless**.
2. If your cluster is available, select it and continue to the next cell. If the cluster is not shown:
   - In the drop-down, select **More**.
   - In the **Attach to an existing compute resource** pop-up, select the first drop-down. You will see a unique cluster name in that drop-down. Please select that cluster.

**NOTE:** If your cluster has terminated, you might need to restart it in order to select it. To do this:

1. Right-click on **Compute** in the left navigation pane and select *Open in new tab*.
2. Find the triangle icon to the right of your compute cluster name and click it.
3. Wait a few minutes for the cluster to start.
4. Once the cluster is running, complete the steps above to select your cluster.

## A. Classroom Setup

Run the following cell to configure your working environment for this course.

**NOTE:** The `DA` object is only used in Databricks Academy courses and is not available outside of these courses. It will dynamically reference the information needed to run the course.

;

```
%run ./Includes/Classroom-Setup-Common
```

```
Note: you may need to restart the kernel using %restart_python or dbutils.library.restartPython() to use updated packages.


DataFrame[]
```

Viewing the current catalog and schema

```
%sql
select current_catalog(),current_schema()
```

▸ _sqldf: pyspark.sql.dataframe.DataFrame = [current_catalog(): string, current_schema(): string]

Table

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

# B. Reading Data

## 1. Schema Inference

Let's start by creating a DataFrame by reading a directory of CSV files.

```
# Read the customers.csv file with schema inference
customers_df = spark.read.format("csv") \
  .option("header", "true") \
  .option("inferSchema", "true") \
  .load("/Volumes/dbacademy_retail/v01/source_files")
```

▸ customers_df: pyspark.sql.dataframe.DataFrame = [customer_id: string, tax_id: string ... 17 more fields]

```
# Show the inferred schema for customers_df
customers_df.printSchema()
```

```
root
 |-- customer_id: string (nullable = true)
 |-- tax_id: string (nullable = true)
 |-- tax_code: string (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- state: string (nullable = true)
```

```
|-- city: string (nullable = true)
|-- postcode: string (nullable = true)
|-- street: string (nullable = true)
|-- number: string (nullable = true)
|-- unit: string (nullable = true)
|-- region: string (nullable = true)
|-- district: string (nullable = true)
|-- lon: string (nullable = true)
|-- lat: string (nullable = true)
|-- ship_to_address: string (nullable = true)
|-- valid_from: string (nullable = true)
|-- valid_to: string (nullable = true)
|-- units_purchased: string (nullable = true)
|-- loyalty_segment: string (nullable = true)
```

> **NOTE:** If the dataset has a header row, column names are inferred. Without a header, columns are named **_c0**, **_c1**, etc., and must be renamed manually.

```
# Alternatively we can use below command to print schema in linear format
print(customers_df.schema)
```

```
StructType([StructField('customer_id', StringType(), True), StructField('tax_id', StringType(), True), StructField('tax_code', StringType(), True), StructField('customer_name', StringType(), True), StructField('state', StringType(), True), StructField('city', StringType(), True), StructField('postcode', StringType(), True), StructField('street', StringType(), True), StructField('number', StringType(), True), StructField('unit', StringType(), True), StructField('region', StringType(), True), StructField('district', StringType(), True), StructField('lon', StringType(), True), StructField('lat', StringType(), True), StructField('ship_to_address', StringType(), True), StructField('valid_from', StringType(), True), StructField('valid_to', StringType(), True), StructField('units_purchased', StringType(), True), StructField('loyalty_segment', StringType(), True)])
```

## `display()` function

The `display()` function is available in Databricks runtimes to be used within notebooks.

`display()` represents a Spark action, which returns results for a DataFrame in a formatted tabular result window. As this is intended for visual inspection only, results are limited to 10,000 rows or 2 MB, whichever is less. Additional features of the `display()` function include:

- Ability to sort or filter data in the result set
- Ability to download result data to a CSV file
- Ability to profile data using the **+** -> **Data Profile** option
- Ability to visualize data using the **+** -> **Visualization** option

```
# Using Display function to preview the data in the DataFrame
display(customers_df)
```

Table

## 2. Explicitly Defining a Schema

Let's load the same dataset into a DataFrame, this time we will explicitly define the schema.

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType
# or ...
from pyspark.sql.types import *

# Define explicit schema using StructType
customer_schema = StructType([
    StructField("customer_id", IntegerType(), False),
    StructField("tax_id", StringType(), True),
    StructField("tax_code", StringType(), True),
    StructField("customer_name", StringType(), True),
    StructField("state", StringType(), True),
    StructField("city", StringType(), True),
    StructField("postcode", StringType(), True),
    StructField("street", StringType(), True),
    StructField("number", StringType(), True),
    StructField("unit", StringType(), True),
    StructField("region", StringType(), True),
    StructField("district", StringType(), True),
    StructField("lon", DoubleType(), True),
    StructField("lat", DoubleType(), True),
    StructField("ship_to_address", StringType(), True),
    StructField("valid_from", IntegerType(), True),
    StructField("valid_to", IntegerType(), True),
    StructField("units_purchased", IntegerType(), True),
    StructField("loyalty_segment", IntegerType(), True)])

# Read the customers.csv file with explicit StructType schema
customers_structtype_df = spark.read.format("csv") \
  .option("header", "true") \
  .schema(customer_schema) \
  .load("/Volumes/dbacademy_retail/v01/source_files/customers.csv")
```

▸ customers_structtype_df: pyspark.sql.dataframe.DataFrame = [customer_id: integer, tax_id: string ... 17 more fields]

```
# Examine the explicit schema
customers_structtype_df.printSchema()
```

```
|-- customer_id: integer (nullable = true)
|-- tax_id: string (nullable = true)
|-- tax_code: string (nullable = true)
```

```
|-- state: string (nullable = true)
|-- city: string (nullable = true)
|-- postcode: string (nullable = true)
|-- street: string (nullable = true)
|-- number: string (nullable = true)
|-- unit: string (nullable = true)
|-- region: string (nullable = true)
|-- district: string (nullable = true)
|-- lon: double (nullable = true)
|-- lat: double (nullable = true)
|-- ship_to_address: string (nullable = true)
|-- valid_from: integer (nullable = true)
|-- valid_to: integer (nullable = true)
|-- units_purchased: integer (nullable = true)
|-- loyalty_segment: integer (nullable = true)
```

## 3. Using a DDL Schema

Let's define the schema now using a DDL schema for readability.

```
ddl_schema = """
  customer_id INT NOT NULL,
  tax_id STRING,
  tax_code STRING,
  customer_name STRING,
  state STRING,
  city STRING,
  postcode STRING,
  street STRING,
  number STRING,
  unit STRING,
  region STRING,
  district STRING,
  lon DOUBLE,
  lat DOUBLE,
  ship_to_address STRING,
  valid_from INT,
  valid_to INT,
  units_purchased INT,
  loyalty_segment INT
"""
```

```
customers_ddl_df = spark.read.format("csv") \
  .option("header", "true") \
  .schema(ddl_schema) \
  .load("/Volumes/dbacademy_retail/v01/source_files/customers.csv")
```

▸ customers_ddl_df: pyspark.sql.dataframe.DataFrame = [customer_id: integer, tax_id: string ... 17 more fields]

```
customers_ddl_df.printSchema()
```

```
root
 |-- customer_id: integer (nullable = true)
 |-- tax_id: string (nullable = true)
 |-- tax_code: string (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- state: string (nullable = true)
```

```
|-- city: string (nullable = true)
|-- postcode: string (nullable = true)
|-- street: string (nullable = true)
|-- number: string (nullable = true)
|-- unit: string (nullable = true)
|-- region: string (nullable = true)
|-- district: string (nullable = true)
|-- lon: double (nullable = true)
|-- lat: double (nullable = true)
|-- ship_to_address: string (nullable = true)
|-- valid_from: integer (nullable = true)
|-- valid_to: integer (nullable = true)
|-- units_purchased: integer (nullable = true)
|-- loyalty_segment: integer (nullable = true)
```

# C. Writing Data From DataFrames

Now let's write out the contents of a DataFrame to different output locations.

## Writing the contents of a DataFrame to a File System

We can write the contents of our DataFrame to a filesystem in various formats using the `write` and `save` methods as shown here.

```
# Define an output path
parquet_output_volume_path = f"/Volumes/{DA.catalog_name}/{DA.schema_name}/v01/customers_parquet"
```

```
# Write the DataFrame out as Parquet files to a directory
customers_ddl_df.write.format("parquet") \
  .mode("overwrite") \
  .save(parquet_output_volume_path)
```

```
# Show the files in the directory
display(dbutils.fs.ls(parquet_output_volume_path))
```

Table

## Writing the contents of a DataFrame to a Table

We can also save our DataFrame to a table (defined in a catalog and schema in Unity Catalog).

```
# Writing our DataFrame to a new table (this is an action)
customers_ddl_df.write.saveAsTable("customers_ddl_df_table")
```

```
# Alternatively you can use the writeTo method which invokes the DataFrameWriterV2
customers_ddl_df.writeTo(
    f"{DA.catalog_name}.{DA.schema_name}.customers_ddl_df_table"
).createOrReplace()
# you have options to partition the table, as well as append, overwrite or createOrReplace
```

```
%sql
-- Read back the data in the table
SELECT * FROM customers_ddl_df_table
```

▸ _sqldf: pyspark.sql.dataframe.DataFrame = [customer_id: integer, tax_id: string ... 17 more fields]

Table

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

# Summary

1. **Reading Data into DataFrames**:
   - Schema inference can be used for CSV files
   - The schema can also be explicitly defined (preferred) using StructType definitions or using a DDL schema
2. **Writing Data from DataFrames**:
   - DataFrames can be written out to a distributed filesystem (like a Unity Catalog volume)
   - DataFrames can also be written out to tables in Unity Catalog